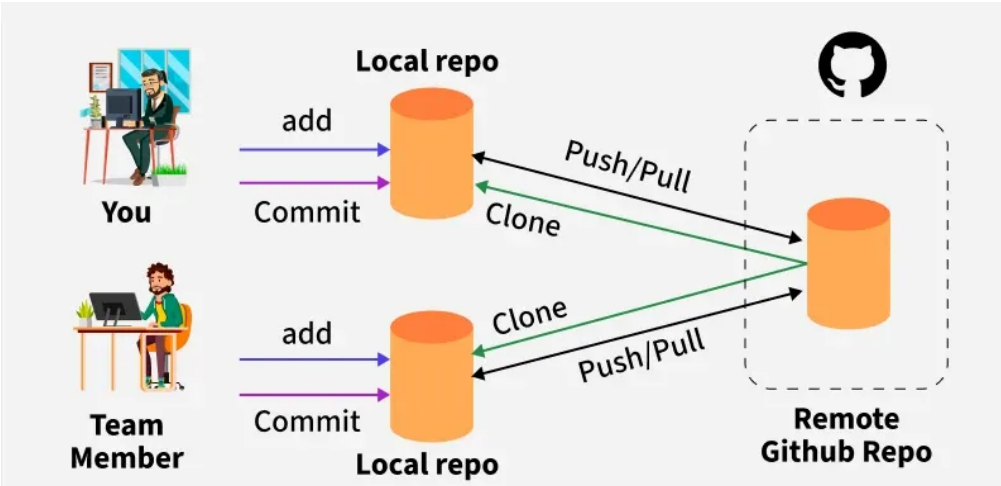
Git & GitHub
for AI Projects

Branching, pull requests, and the one thing that breaks every AI repo: large model files.



`.py` `.ipynb` `.safetensors` `.gguf`

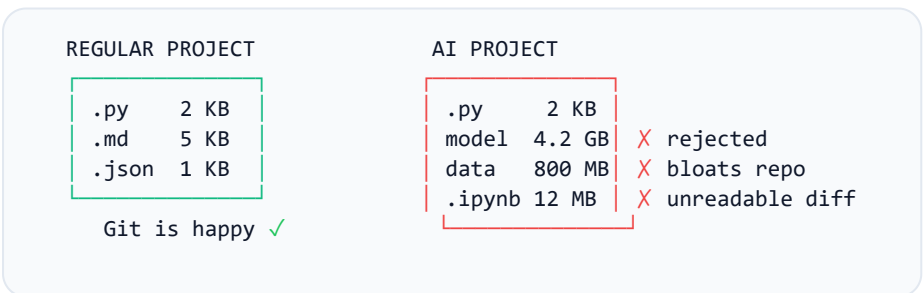
AI Coach John Follow | PROITBRIDGE Strive For Better Future | Repost

⚠ THE PROBLEM

Git was built for text. AI projects are mostly not text.

Git was designed to track small text files and store the full diff history of every change. AI projects violate almost every assumption that design makes.

- ✕ **Model files are huge:** a single .safetensors file can be gigabytes; GitHub rejects >100 MB
- ✕ **Binary files can't be diffed:** Git stores a full copy on every change. 10 edits to a 2 GB file = 20 GB repo
- ✕ **Notebooks are a nightmare:** .ipynb files store execution outputs, making meaningless diffs
- ✕ **Secrets leak constantly:** .env files with API keys get committed

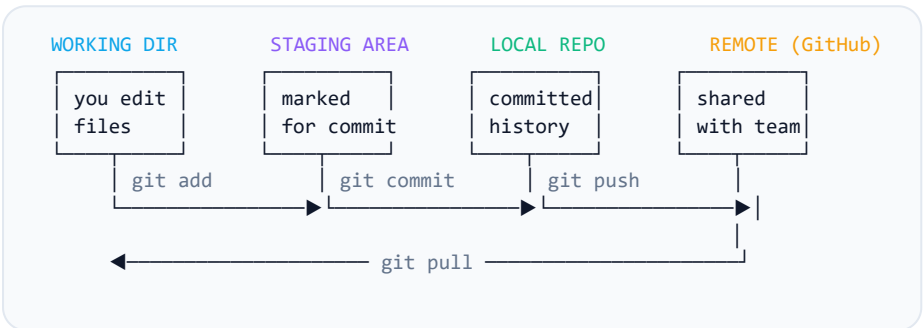


AI Coach John Follow | PROITBRIDGE Strive For Better Future | Repost

MENTAL MODEL

Four places your code can live

Almost every Git confusion comes from not knowing which of these four areas a file is currently in.




```
git status              # which area is each file in?
git add train.py        # working dir  → staging
git commit -m "..."     # staging      → local repo
git push                # local repo   → GitHub
git pull                # GitHub       → local repo + working
```

AI Coach John Follow | PROITBRIDGE Strive For Better Future | Repost

SETUP

The first five minutes that save you weeks

```
mkdir rag-service && cd rag-service
git init

# CREATE .gitignore FIRST (before any git add)
cat > .gitignore << 'EOF'
# Secrets
.env
*.key
# Environments
.venv/
__pycache__/
# Data & models (handled separately)
data/raw/
*.safetensors
# Notebook noise
.ipynb_checkpoints/
EOF

git add .gitignore
git commit -m "chore: initial project setup"
```

ORDER MATTERS

✓ .gitignore → git init → git add → commit
(clean history)

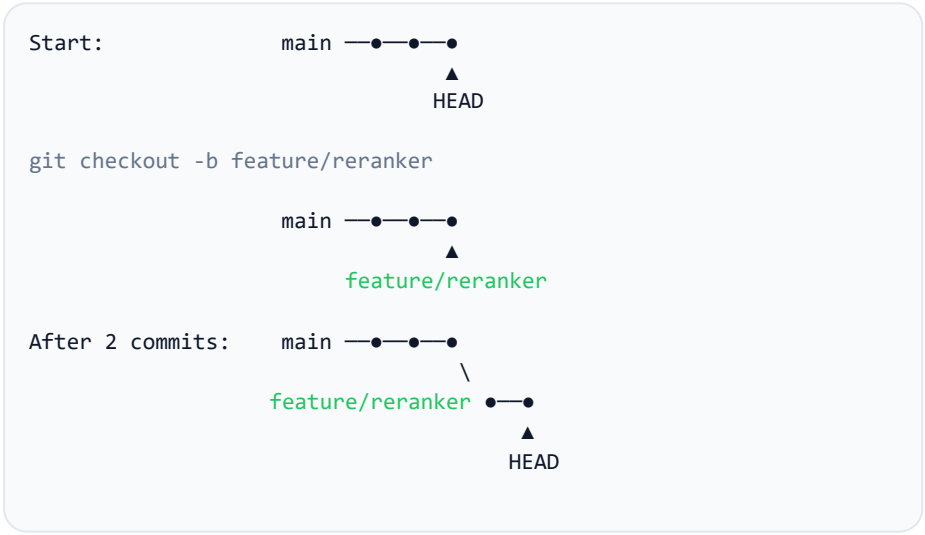
✗ git add → commit → .gitignore
(secret is now permanent history)


AI Coach John Follow | PROITBRIDGE Strive For Better Future Repost

BRANCHING

A branch is just a movable pointer to a commit

Branches are cheap. Nothing is copied. Git simply creates a new label that moves forward as you commit.




```
git checkout -b feature/reranker    # create + switch
git branch                          # list branches
git checkout main                   # switch back
git branch -d feature/reranker      # delete when done
```


AI Coach John Follow | PROITBRIDGE Strive For Better Future | Repost

CONVENTIONS

A branch naming convention that makes your repo readable

```
feature/    new capability
fix/        bug fix
exp/        experiment (may die)
data/       dataset changes
eval/       evaluation work
chore/      tooling, config, deps
```

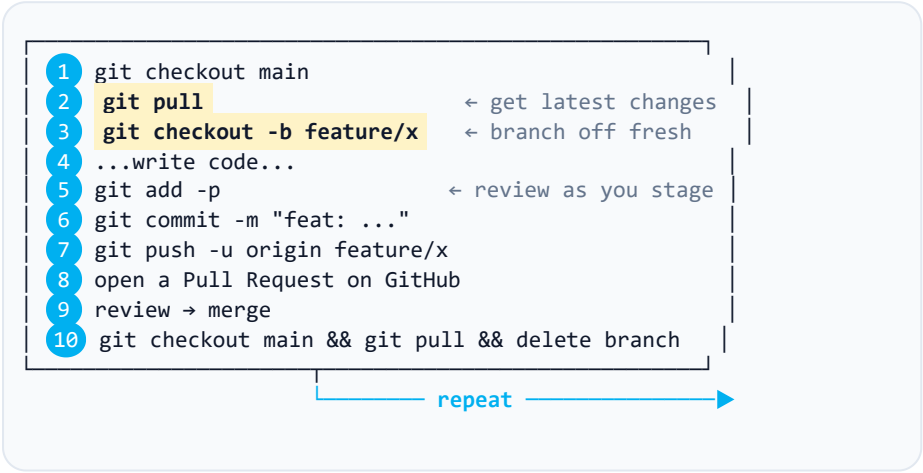
Why `exp/` matters specifically for AI work: Most AI experiments fail. A dedicated prefix makes it socially and technically obvious that a branch may be abandoned, so nobody feels obliged to merge a dead end.



AI Coach John Follow | PROITBRIDGE Strive For Better Future | Repost

WORKFLOW

The everyday loop you'll run dozens of times




```
git add -p      # stage hunk-by-hunk. catches debug
                # prints & secrets before commit

git log --oneline --graph --all  # visualise branches
```


AI Coach John Follow | PROITBRIDGE Strive For Better Future | Repost

COMMITS

Commit Messages That Actually Help

```
<type>: <what changed and why>

feat:     add cross-encoder reranking step
fix:      correct chunk overlap calculation
exp:      bge-small beats e5-base by 4% recall
data:     add 2k labelled support tickets
eval:     add faithfulness metric to CI
perf:     enable flash attention, 1.8x faster
chore:    pin langchain to 0.2.11
```

BAD

"updates"

"fixed stuff"

"wip"

"asdf"

GOOD

"feat: add hybrid BM25..."

"fix: chunk overlap 50..."

"exp: e5-base vs bge..."

"perf: batch embeddings"

Why `exp:` matters: Six months later, "exp: bge-small beats e5-base" saves someone from re-running the same comparison.



AI Coach John Follow | PROITBRIDGE Strive For Better Future

Repost

THE BIG ONE

Why Model Files Break Git

Git stores a **complete snapshot** of a binary file on every change. It cannot compute meaningful diffs on binary data.

```
COMMITTING A 2 GB MODEL, EDITED 5 TIMES

commit 1   ▒▒▒▒▒▒▒▒▒▒  2 GB
commit 2   ▒▒▒▒▒▒▒▒▒▒  2 GB   (full copy again)
commit 3   ▒▒▒▒▒▒▒▒▒▒  2 GB   (full copy again)
commit 4   ▒▒▒▒▒▒▒▒▒▒  2 GB   (full copy again)
commit 5   ▒▒▒▒▒▒▒▒▒▒  2 GB   (full copy again)
──────────────────────────
.git dir   = 10 GB  ⚠

Every clone downloads ALL of it. Forever.
```

GitHub Limits:

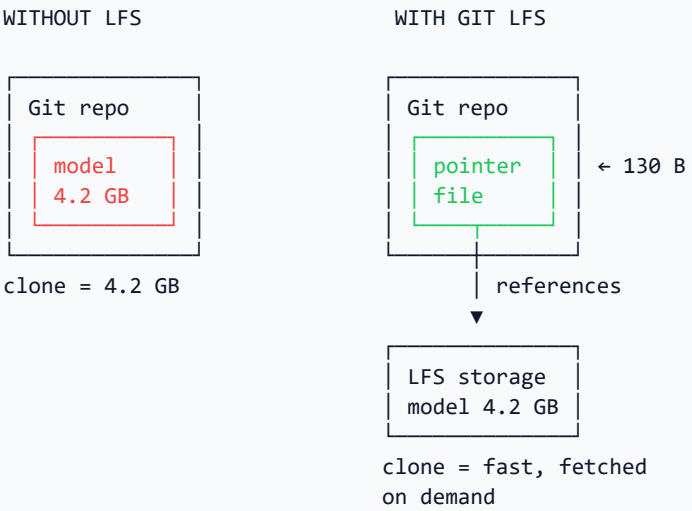
```
> 50 MB  ⚠ warning
> 100 MB ✕ REJECTED
```

The trap: deleting the file in a later commit does not shrink the repo. The blob stays in history permanently.

GIT LFS

Store a tiny pointer in Git, the real file elsewhere

Git LFS (Large File Storage) replaces big files in your repo with small text pointers, storing actual bytes on an LFS server.



INSIDE THE POINTER FILE (130 BYTES):

```
version https://git-lfs.github.com/spec/v1
oid sha256:4d7a214614ab2935c943f9e...
size 4294967296
```

AI Coach John Follow PROITBRIDGE Strive For Better Future Repost

SETUP GIT LFS

Step by step, with the ordering that matters

```
# 1. Install (once per machine)
git lfs install

# 2. Tell LFS which patterns to track
git lfs track "*.safetensors"
git lfs track "*.gguf"

# 3. COMMIT .gitattributes FIRST
git add .gitattributes
git commit -m "chore: track models with LFS"

# 4. NOW add your model files
git add models/adapter.safetensors
git commit -m "feat: add adapter"
git push
```

CORRECT ORDER

`lfs track → commit .gitattributes → add model`

WRONG ORDER

```
add model → commit → lfs track
└── file is already a normal Git blob in history
```

AI Coach John Follow | PROITBRIDGE Strive For Better Future | Repost

TRAPS

Git LFS: Limits & Common Traps

```
GITHUB LFS FREE TIER
Storage     ▓▓▓▓▓▓▓▓▓▓  1 GB
Bandwidth   ▓▓▓▓▓▓▓▓▓▓  1 GB / month
          ▲ bandwidth is consumed on every
            clone AND CI run
```

- **Bandwidth burns fast:** Cache LFS in CI.
- **Tracking late does nothing:** It's already a blob.
- **git lfs pull needed:** If a model file is 130 bytes, you only got the pointer.

Is this file...	Action
small adapter you produced?	✓ Git LFS
base model from HF?	✗ download in Dockerfile
large training dataset?	✗ DVC / S3
small eval set (< 5 MB)?	✓ plain Git

DVC

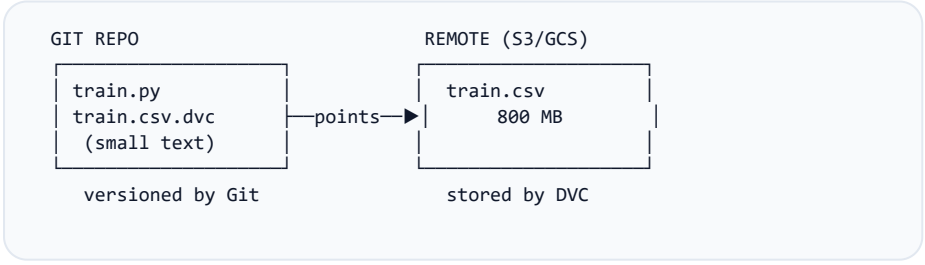
Beyond LFS: Versioning Datasets with DVC

For large datasets, DVC keeps a Git-tracked *metadata file* while data lives in S3/GCS.

```
dvc init
# Creates data/train.csv.dvc (small text file)
dvc add data/train.csv

git add data/train.csv.dvc data/.gitignore
git commit -m "data: track training set v1"

# Point DVC at remote storage
dvc remote add -d storage s3://my-bucket
dvc push        # upload the actual data
```



> **`git checkout <old-commit> && dvc pull` restores the EXACT dataset from that point in time.**

AI Coach John Follow | PROITBRIDGE Strive For Better Future | Repost

NOTEBOOKS

The other file type that ruins AI repos

A notebook stores outputs, base64 images, and execution counts. Running a cell changes the file even if code didn't.

```
You re-run one cell, change nothing:
git diff
─────────────────────────────────────
- "execution_count": 12,
+ "execution_count": 13,
- "image/png": "iVBORw0KGgoAAA..." ← 400 KB base64
+ "image/png": "iVBORw0KGgoAAA..." ← different 400 KB
... 2,847 more lines ...

Reviewer: "what actually changed?"
```

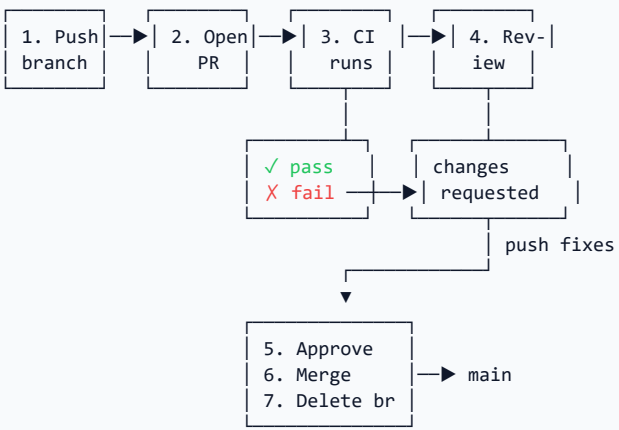
THE FIX: STRIP OUTPUTS AUTOMATICALLY

```
pip install nbstripout
nbstripout --install  # sets up a git filter
```

Outputs are stripped on commit, but stay visible in your local notebook.

PRS

How work actually gets merged




```
# After review comments, push to the SAME branch:
git add .
git commit -m "fix: address review feedback"
git push    # the PR updates automatically
```

AI Coach John Follow | PROITBRIDGE Strive For Better Future | Repost

DESCRIPTIONS

A PR template reviewers will thank you for

What changed
Replaced naive top-k retriever with hybrid search.

Evaluation

Metric	Before	After
Context recall	0.68	**0.81**
p95 latency	420ms	510ms

Ran on the 200-question eval set (`eval/questions_v2.jsonl`).

AI PRs need one extra section: Evaluation Numbers. "It feels better" is not reviewable. A metrics table is.

CODE REVIEW

Reviewing an AI PR: The Checklist

- [ ] Evaluation numbers included?
- [ ] Eval run on held-out set?
- [ ] Any hardcoded API keys leaked?
- [ ] Model/data files added without LFS?
- [ ] Notebook outputs stripped?
- [ ] Config values not hardcoded inline?

QUICK CHECK BEFORE APPROVING

```
git diff main --stat
──────────────────────────────────────
src/retrieval.py     | 84 +++++---
models/adapter.bin   | Bin 0 -> 4.2GB ⚠
notebooks/eda.ipynb | 2847 +++++++++ ⚠
```

EMERGENCIES

Committed Something You Shouldn't Have?

```
# SCENARIO 1: Not pushed yet
git reset HEAD~1            # undo commit, KEEP changes
git reset --hard HEAD~1     # undo commit, DISCARD changes

# SCENARIO 2: Huge file, not pushed
git rm --cached models/big_model.safetensors
echo "*.safetensors" >> .gitignore
git commit --amend -C HEAD
```

SCENARIO 3: PUSHED A SECRET

1. REVOKE THE KEY IMMEDIATELY at the provider.

2. Clean history: `git filter-repo ...`

3. Force push & notify team to re-clone

Deleting the file in a new commit does **NOT** help. Bots scrape leaked keys within minutes.


AI Coach John Follow | PROITBRIDGE Strive For Better Future Repost

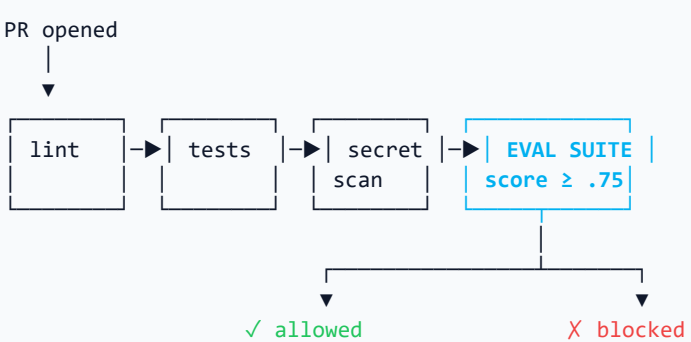
CI / CD

CI that runs your evals, not just tests

```yaml
# .github/workflows/ci.yml
jobs:
  test:
    steps:
      - uses: actions/checkout@v4
        with:
          lfs: false  # skip LFS to save bandwidth!

      - name: Unit tests
        run: pytest tests/

      - name: Run evaluation suite
        env:
          API_KEY: ${{ secrets.API_KEY }}
        run: python run_eval.py --threshold 0.75
```



ARCHITECTURE

The Complete AI Repo Structure

```
rag-service/
├── .gitignore              # secrets, envs, raw data
├── .gitattributes          # Git LFS tracking rules
├── .env                    ✗ never committed
├── .env.example            ✓ committed template
├── src/                    ✓ plain Git
├── models/
│   └── adapter.safetensors ✓ Git LFS
├── data/
│   ├── raw/                ✗ gitignored
│   ├── eval_set.jsonl      ✓ plain Git (small)
│   └── train.csv.dvc       ✓ DVC pointer
└── notebooks/
    └── explore.ipynb       ✓ nbstripout applied
```

WHICH TOOL FOR WHICH FILE	
small text/code & eval sets	plain Git
model adapters	Git LFS
large datasets	DVC + object storage
base models	download at build time
secrets	.env (gitignored)

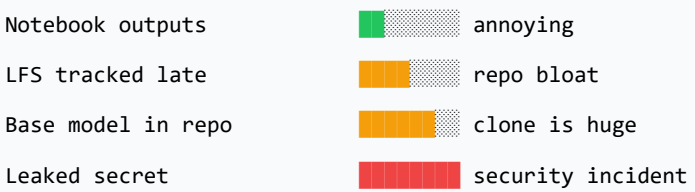
AI Coach John Follow | PROITBRIDGE Strive For Better Future | Repost

MISTAKES

What breaks AI repos most often

- .env before .gitignore
- LFS track AFTER commit
- Notebooks with outputs
- Base models in repo
- Force pushing main
- CI cloning LFS every run
- Deleting a leaked secret
- PRs without eval numbers

COST OF EACH MISTAKE



AI Coach John Follow | PROITBRIDGE Strive For Better Future Repost

Git wasn't built for AI, but it works if routed correctly

Branch for every change, keep main deployable, put eval numbers in every PR, and send each file type to the right place.

code	Git
adapters	Git LFS
datasets	DVC
base models	download
secrets	.env (gitignored)

Save this reference. Follow for more AI engineering breakdowns.


AI Coach John Follow | PROITBRIDGE Strive For Better Future Repost